

Cifrado César con Divide y Vencerás y Programación Dinámica

Análisis de Algoritmos

Juan Ernesto Lopez Arce Delgado



Equipo César - D01

Aurelio Rendon Viciego - ICOM

Humberto de Jesus Pena Duenas - ICOM

Ricardo Alejandro Rosas Arreola - ICOM

Domingo 26 de octubre del 2025

· **Introducción**

Para este ejercicio el equipo decidió seguir con el mismo algoritmo con el que habíamos trabajado desde la aplicación de la fuerza bruta, este es el *Cifrado César*.

El Cifrado Cesar es un algoritmo de encriptación basado en una sencilla fórmula que aplica “saltos” o “desplazamientos” en cada letra del mensaje volviéndolo imposible de leer/entender a simple vista.

Para la aplicación de este método en el algoritmo tomamos como inspiración base un concepto clave de la película “The imitation game (2014)” para el descifrado del mensaje. La idea con la que terminan descifrando los mensajes de la película es encontrar o definir una palabra base que se supone sabemos que está en el texto cifrado. En este caso la palabra clave la define el programa.

· **Objetivo**

Al igual que el anterior ejercicio en el que se aplicó la técnica de *Fuerza Bruta*, el objetivo de este no es el cifrado, si no el descifrado del mensaje proporcionado.

La idea es tener un texto base ya sea algún mensaje o alguna palabra, este texto se cifrara como ya se ha venido haciendo, asignando un salto aleatorio y aplicándolo a todas las letras del mensaje para después poder seguir ciertas instrucciones que nos llevará al mensaje descifrado. En la sección del desarrollo de la actividad en este reporte se entrará más a detalle acerca del procedimiento.

A diferencia del anterior ejercicio, en este algoritmo se planea obtener el descifrado completo de todo el mensaje y presentarlo de esa manera al usuario ya que en el anterior se imprimían los 26 posibles mensajes descifrados sin indicar cual seria el verdadero texto descifrado.

Desarrollo

Este programa sigue sencillos pasos que se definen con el objetivo de lograr una sencilla comprensión de funcionalidad.

Al iniciar el programa vamos a encontrarnos con la llamada a la función **generar_semilla_caotica()** que lo que hace es agregar una capa más a la “aleatoriedad” que se espera en los siguientes pasos del programa tanto para cifrar como para definir una palabra guía y así evitar que las ejecuciones sigan algún patron.

Python

```
def generar_semilla_caotica():
    # 1. Fuente: Hora actual en nanosegundos (muy volátil)
    fuente1 = str(time.perf_counter_ns()).encode('utf-8')
    # 2. Fuente: ID del Proceso (cambia con cada ejecución)
    fuente2 = str(os.getpid()).encode('utf-8')
    # 3. Fuente: Dirección de memoria de un nuevo objeto (volátil)
    fuente3 = str(id(object())).encode('utf-8')

    print("--- Mejorando Semilla (sin 'secrets') ---")
    print(f"Fuente 1 (Tiempo ns): {fuente1.decode()}")
    print(f"Fuente 2 (Process ID): {fuente2.decode()}")
    print(f"Fuente 3 (Memoria ID): {fuente3.decode()}")

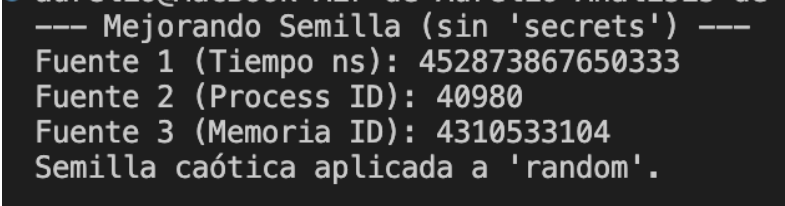
    # 4. Combinar y Hashear todas las fuentes juntas
    hash_object = hashlib.sha256()
    hash_object.update(fuente1)
    hash_object.update(fuente2)
    hash_object.update(fuente3)

    hash_digest = hash_object.digest()

    # 5. Convertir el hash final en un entero
    semilla_int_final = int.from_bytes(hash_digest, 'big')

    # 6. Aplicar esta semilla caótica al módulo 'random'
    random.seed(semilla_int_final)
    print("Semilla caótica aplicada a 'random'.\n")
```

Con la ejecución de esta sección del código esperaremos una salida en pantalla similar a la mostrada en la siguiente imagen:



```
--- Mejorando Semilla (sin 'secrets') ---
Fuente 1 (Tiempo ns): 452873867650333
Fuente 2 (Process ID): 40980
Fuente 3 (Memoria ID): 4310533104
Semilla caótica aplicada a 'random'.
```

En la ejecución del programa nos vamos a encontrar la función **cifrar()** que sirve para cifrar el mensaje, lo que hace es definir un número aleatorio en un rango de 1 a 26 que va a representar el desplazamiento que se va a aplicar al mensaje. El mensaje sin cifrar (**FRASE**), el mensaje cifrado(**FRASE_CIFRADA**) y la palabra guía (**PALABRA_GUIA**) las vamos a tener definidas en las variables globales del programa.

Python

```
def cifrar():
    global PALABRA_GUIA, FRASE_CIFRADA
    cifrada = []
    # Generar salto_aleatorio (desplazamiento de César)
    salto = random.randint(1, 25)
    fraseMinuscula = FRASE.lower()
    # Seleccionar PALABRA_GUIA con selectGuia()
    PALABRA_GUIA = selectGuia(fraseMinuscula)

    # Recorrer cada carácter de la frase original
    for letra in fraseMinuscula:
        char = ord(letra)
        if char != 32: # Para espacios: copiar tal cual
            # Para letras: aplicar desplazamiento salto_aleatorio
            if char == 241 or char == 209: # Caso especial para 'ñ'
                char = definirChar(110 + salto)
            else:
                char = definirChar(char + salto)
        cifrada.append(chr(char)) # Añadir carácter a lista cifrada

    FRASE_CIFRADA = "".join(cifrada) # Unir lista en FRASE_CIFRADA
```

Dentro de la función **cifrar()** que se muestra en el fragmento de código anterior nos podemos percatar que hay llamadas a dos funciones auxiliares, estas son **definirChar()** y **selectGuia()**, vamos a explicarlas a continuación:

selectGuia() es la función que define esa palabra que nos va a guiar para el descifrado del mensaje más adelante. Recibe la frase sin cifrar y devuelve una palabra elegida con el método `random.choice()`

Python

```
def selectGuia(frase):
    palabra = ""
    while len(palabra) < 3:
        lista_palabras = frase.split() # Separa la frase en un arreglo de palabras
        palabra = random.choice(lista_palabras) # Elige un elemento del arreglo al azar
    return palabra
```

definirChar() es una función que se va a utilizar frecuentemente a lo largo de todo el programa ya que es la que nos ayuda a “dar la vuelta” al abecedario en caso de requerir un salto en alguna letra que requiera volver a empezar de nuevo.

Python

```
def definirChar(ascii):  
    if ascii >= 123:      # Wrap-around si pasa de 'z'  
        return ascii - 122 + 96  
    if ascii < 97:        # Wrap-around si es menor que 'a'  
        return 122 - (96 - ascii)  
    return ascii
```

Y la última llamada que nos vamos a encontrar en la ejecución del programa es la llamada a `decifradoCesar()`, la función principal más importante de todo el ejercicio ya que en esta aplicamos la técnica de divide y vencerás y la programación dinámica, vamos a analizarla por partes.

En esta función es en donde empezamos a trabajar con la palabra guía, la cual no tiene sentido comparar con absolutamente todas las palabras del cifrado, si no solo con aquellas que cumplan con el largo de palabras de esta.

Python

```
wordsWithLen = []  
for palabra in FRASE_CIFRADA.split(): #recorre el mensaje cifrado palabra por palabra  
    if len(palabra) == len(PALABRA_GUIA):  
        wordsWithLen.append(palabra) # Al hayar una palabra con el largo, la guarda
```

El siguiente paso es simple, lo que hace es trabajar con cada palabra guardada en el arreglo “wordsWithLen”. Primero calcula el salto que hay entre la primera letra de la palabra del arreglo y la primera letra de la palabra guía para después aplicar ese salto letra por letra al resto de las letras en la palabra cifrada y compararlas con el índice en el mismo índice de letra en la palabra guía. A lo largo de las comparaciones usamos una bandera que nos indica si la coincidencia del desplazamiento sigue, en caso de no coincidir, la bandera se vuelve negativa y pasamos a la siguiente palabra

Python

```
# Para cada palabra en wordsWithLen:  
for palabra in wordsWithLen:  
    if not palabra or not PALABRA_GUIA:
```

```

        continue

    # Calcular salto (basado en primera letra)
    salto = ord(palabra[0]) - ord(PALABRA_GUIA[0])
    if salto < 0:
        salto += 26

    # Asumimos que coincide hasta que se demuestre lo contrario
    coincidencia_total = True

    # Iterar cada letra de la palabra guía
    for i in range(len(PALABRA_GUIA)):
        letra_cifrada = palabra[i]
        letra_guia_original = PALABRA_GUIA[i]

        char_descifrado = definirChar(ord(letra_cifrada) - salto)

        if chr(char_descifrado) != letra_guia_original:
            coincidencia_total = False
            break

    if coincidencia_total:
        desplazamiento = salto
        break

```

El equipo pensó en aplicar la programación dinámica haciendo un diccionario de las letras desplazadas y así evitar cálculos de desplazamientos repetitivos en la frase cifrada.

Python

```

# Si desplazamiento es diferente a None: descifrar texto completo
diccionario = {}
for i in range(97, 123): # Crear diccionario de descifrado
    diccionario[chr(i)] = chr(definirChar(i - desplazamiento))

```

El último paso de la ejecución del programa es la aplicación del desplazamiento en todo el texto cifrado, logrando así el descifrado del mensaje

Python

```
# Aplicar descifrado a toda la frase
texto_descifrado = ""
for letra in FRASE_CIFRADA:
    if letra == " ": # Respetar espacios
        texto_descifrado += " "
    elif letra in diccionario:
        texto_descifrado += diccionario[letra]
    else:
        texto_descifrado += letra
```

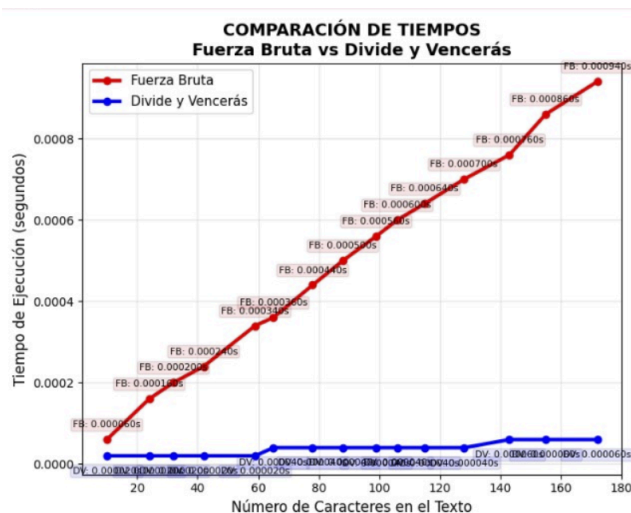
La ejecución del programa nos deja una impresión en la terminal con el estilo de la mostrada en la siguiente imagen:

```
Texto cifrado: mv cv tcoiz lm ti uivkpi lm kcgw vwujzm vw ycqmzw ikwzlizum vw pi uckpw bqmuxw ycm dqd
qi cv pqlitow lm twa lm tivhi mv iabqttmzw ilizoi ivbqoci zwkqv ntikw g oitow kwzzmlwz
Palabra guia: que
Desplazamiento encontrado: 8
Texto descifrado: en un lugar de la mancha de cuyo nombre no quiero acordarme no ha mucho tiempo que
vivía un hidalgo de los de lanza en astillero adarga antigua rocin flaco y galgo corredor
```

Conclusiones

Este ejercicio fue bien interesante ya que logramos descifrar por completo la frase cifrada, cosa que también hacíamos en el anterior pero mezclada con basura o descifrados con desplazamientos inválidos.

Haciendo una comparativa de tiempos de ejecución con el anterior algoritmo implementado nos podemos encontrar con una gráfica como la mostrada en las siguientes imágenes:



RESUMEN FINAL - COMPARACIÓN DE TIEMPOS		
Longitud	FB Tiempo (s)	DV Tiempo (s)
10	0.000060	0.000020
24	0.000160	0.000020
32	0.000200	0.000020
42	0.000240	0.000020
59	0.000340	0.000020
65	0.000360	0.000040
78	0.000440	0.000040
88	0.000500	0.000040
99	0.000560	0.000040
106	0.000600	0.000040
115	0.000640	0.000040
128	0.000700	0.000040
143	0.000760	0.000060
155	0.000860	0.000060
172	0.000940	0.000060

Analizando la imagen nos podemos percatar de que en efecto el algoritmo de fuerza bruta requiere muchísimo más tiempo de ejecución en base al número de caracteres en el texto ya que aplica los 26 posibles desplazamientos a todo el texto cifrado para imprimirlos en pantalla e inferir que el usuario va a detectar el texto descifrado lo cual nos complica en textos con muchos caracteres.

A comparación del de Fuerza Bruta, el algoritmo que implementa Divide y Vencerás aplica un bajo tiempo de ejecución a pesar de la cantidad de caracteres que tenga el texto ya que solo trabaja una vez con el texto completo para obtener las palabras con que coincidan con el largo de la palabra guía, y una vez por palabra en esta lista de palabras que coinciden.

Si nos ponemos a analizar nos podemos dar cuenta que el peor de los casos en este algoritmo es en el que todas las palabras del texto tengan el mismo largo, como en la frase “que feo ese mal rey oso” cuyas palabras siguen el patrón de largo de 3. El mejor de los casos es obviamente aquel texto cuya frase guía sea la única de ese largo en todo el texto.